BEAM: Encoding-Aware Greedy Merging for Scalable Flat Graph Summarization

Boyang Zhang
Hunan University
Changsha, Hunan, China
zhangboyang@hnu.edu.cn

Guoqing Xiao
Hunan University
Changsha, Hunan, China
xiaoguoqing@hnu.edu.cn

Yuedan Chen
Central South University
Changsha, Hunan, China
chenyuedan@hnu.edu.cn

Zhuo Tang
Hunan University
Changsha, Hunan, China
ztang@hnu.edu.cn

Xiaofei Zhang
University of Memphis
Memphis, Tennessee, USA
Xiaofei.Zhang@memphis.edu

Kenli Li
Hunan University
Changsha, Hunan, China
lkl@hnu.edu.cn

ABSTRACT

Flat correction-set graph summarization represents a graph with supernodes, superedges, and correction edges, enabling exact reconstruction with a compact flat summary. Scalable methods in this model follow an iterative divide–merge–encode pipeline, and the repeated merge stage is the main scalability challenge because candidate pairs must be generated, evaluated under the encoding objective, selected without conflicts, and applied while the summary state evolves.

We present BEAM, an encoding-aware greedy merging algorithm for scalable flat graph summarization. BEAM makes flat summarization faster by decoupling candidate discovery from objective-faithful merge scoring: lightweight profiles only prune the search space, while retained candidates are scored by exact local correction-set gain. Concretely, BEAM builds a sparse candidate graph within each divide group, applies greedy positive-gain matching within groups, and defers partition updates to the end of each group block. Retained gain evaluations are packed into batched scoring tasks, with CUDA used as one backend for that scoring step.

On ten common datasets, BEAM is faster than SWeG on all datasets and faster than SLUGGER on nine of them, with geometric mean speedups of 2.69× over SWeG and 2.29× over SLUGGER. BEAM also produces smaller relative output size than SWeG on all ten common datasets, while compression results against the hierarchical SLUGGER baseline remain mixed. Runtime breakdowns of the batch-ea-blocked CUDA implementation show that merge-related work accounts for 50.0%–97.9% of BEAM's execution time, confirming that BEAM targets a major, and often dominant, cost in the flat summarization pipeline.

PVLDB Reference Format:
Boyang Zhang, Guoqing Xiao, Yuedan Chen, Zhuo Tang, Xiaofei Zhang, and Kenli Li. BEAM. PVLDB, TBD(TBD): TBD, Under review.
doi:TBD

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/githubofzby/beam.

Proceedings of the VLDB Endowment, Vol. TBD, No. TBD ISSN 2150-8097.
doi:TBD

1 INTRODUCTION

Large graphs are now routinely used to represent web pages, social interactions, communication records, biological relations, knowledge bases, and many other forms of relational data. Directly storing and processing such graphs is expensive because graph analytics repeatedly access large, sparse, and irregular adjacency structures. A broad body of graph compression work therefore seeks to reduce storage cost through vertex ordering, reference encoding, interval encoding, locality-aware layout, recursive partitioning, and compressed graph indexes [1, 3, 5, 8, 9]. These techniques primarily encode a fixed input graph compactly. Graph summarization takes a complementary approach: it constructs a smaller structural representation that groups vertices and records the information needed to reconstruct or approximate the original graph [17].

This paper focuses on flat correction-set graph summarization, where a graph is represented by disjoint supernodes, superedges, and positive and negative correction edges [19, 21]. This model has simple reconstruction semantics and can be used as a compact preprocessing representation for large graphs. Scalable methods for this model commonly follow an iterative divide–merge–encode pipeline: divide groups the current supernodes, merge chooses beneficial supernode pairs, and encode materializes the final superedges and corrections.

The repeated merge stage is the main scalability challenge. It is executed over many iterations while the summary state is changing. Each accepted merge changes supernode memberships, local neighborhoods, future candidate pairs, and the encoding costs used by later decisions. As a result, a scalable method must restrict candidate search, but merge decisions should still reflect the correction-set objective rather than a lightweight similarity proxy alone. In particular, high profile similarity does not imply positive encoding gain, because the benefit of a merge depends on the affected local pair costs under the current partition. This creates a tension between scalability and objective fidelity.

BEAM resolves this tension by pruning candidates cheaply but scoring retained merges exactly inside the same flat correction-set model. It uses lightweight profile-based screening to construct a sparse candidate graph, evaluates retained candidates by exact local correction-set gain, selects positive-gain non-conflicting merges

within groups, and defers partition updates at block granularity. This organization exposes a batched scoring workload without changing the underlying summary model.

The same separation also clarifies the role of parallel execution. BEAM treats batched gain evaluation as the GPU-amenable component of the flat summarization pipeline. Grouping, candidate generation, matching, merge application, and final encoding remain part of the host-side pipeline, while retained exact-gain computations can be packed into batched CPU/GPU scoring tasks. BEAM accelerates repeated candidate scoring, while leaving grouping, matching, and partition updates on the host.

BEAM also differs from approaches that improve summarization by changing the representation. SLUGGER, for example, introduces hierarchical graph summarization with nested supernodes and positive and negative edges between hierarchical supernodes [14]. BEAM keeps the flat correction-set representation fixed. This choice makes SWeG the primary same-model baseline and makes the comparison with SLUGGER a cross-model reference point.

We therefore ask whether the repeated merge stage can be reorganized so that exact encoding-aware scoring is applied only to a sparse set of promising candidates, while the final output remains a standard flat correction-set summary.

This paper makes the following contributions.

- We decompose scalable flat correction-set graph summarization into divide, merge, and encode stages, and identify the repeated merge stage—candidate generation, encoding-aware gain evaluation, conflict-free selection, and partition update—as the dominant scalability bottleneck.
- We propose BEAM, which decouples cheap candidate discovery from objective-faithful merge scoring: profiles only sparsify the search space, and retained candidates are selected by exact local correction-set gain with greedy positive-gain matching.
- We show that the gain of each retained candidate merge can be computed from its affected local neighborhood rather than by re-encoding the full summary, enabling batched evaluation of retained candidate scores.
- We evaluate BEAM against SWeG as a same-model flat baseline and SLUGGER as a hierarchical contrast baseline, and we separately characterize the cost of the CUDA scoring backend used for batched retained-candidate evaluation.

2 BACKGROUND AND PROBLEM SETTING

2.1 Flat Correction-Set Graph Summarization

Let the input be a simple undirected graph $G = (V, E)$. A flat graph summary replaces the original vertices with a partition $\mathcal{S} = \{S_1, \ldots, S_r\}$, where each $S_i \subseteq V$, $S_i \cap S_j = \emptyset$ for $i \neq j$, and $\bigcup_i S_i = V$. Each set S_i is called a supernode. The partition is flat because every original vertex belongs to exactly one supernode and the summary does not maintain nested or overlapping supernodes. This representation is the basis of correction-set graph summarization and is also the representation used by scalable flat methods such as SWeG [19, 21].

Given a flat partition, the summary stores a set of superedges P together with two correction sets C^+ and C^-. A superedge between two supernodes represents the decision to encode the corresponding vertex-pair region densely at the summary level. Positive corrections C^+ store original edges that are not implied by the selected superedges, while negative corrections C^- store missing edges that would otherwise be implied by those superedges. We write the resulting summary as

$$S = (\mathcal{S}, P, C^+, C^-).$$

This representation has explicit reconstruction semantics: the original graph can be recovered by expanding the superedges and then applying the positive and negative corrections.

Table 1: Frequently used notation.

Symbol	Meaning
$G = (V, E)$	Input graph.
$\mathcal{S}$	Current flat partition into supernodes.
X, Y, Z	Current supernodes.
P	Superedge set in the final flat summary.
C^+, C^-	Positive and negative correction sets.
W_X	Sparse neighborhood profile of supernode X.
Q	Divide group of active supernodes.
C_Q	Candidate pair set inside group Q.
C_{pos}	Positive-gain candidate list.
M_{XY}	Merged supernode $X \cup Y$.
$\mathcal{M}_Q$	Conflict-free merge set selected inside group Q.
$\text{cap}(X, Y)$	Number of possible original edges represented by pair (X, Y).
$L(X, Y)$	Local correction-set encoding cost for pair (X, Y).
$\text{gain}(X, Y)$	Exact local encoding gain of merging X and Y.

For two current supernodes X and Y, let $e(X, Y)$ be the number of original graph edges represented by the pair. The number of possible undirected edges covered by the pair is

$$\text{cap}(X, Y) = \begin{cases} |X||Y|, & X \neq Y, \\ |X|(|X| - 1)/2, & X = Y. \end{cases} \tag{1}$$

The encoder has two local choices. It can represent the pair sparsely by storing existing edges directly:

$$L_{\text{sparse}}(X, Y) = e(X, Y). \tag{2}$$

Alternatively, it can represent the pair densely by storing one superedge and then recording missing edges as negative corrections:

$$L_{\text{dense}}(X, Y) = 1 + \text{cap}(X, Y) - e(X, Y), \tag{3}$$

where the constant term accounts for the superedge itself. The local correction-set cost is therefore

$$L(X, Y) = \min\{L_{\text{sparse}}(X, Y), L_{\text{dense}}(X, Y)\}. \tag{4}$$

This pairwise encoding rule is the flat correction-set objective used in this paper. BEAM keeps this representation family unchanged: its output is still a flat partition with superedges and correction edges, rather than a hierarchical summary.

Table 1 summarizes the notation used throughout the paper.

2.2 Divide-and-Merge Construction

Flat correction-set summaries are commonly built through an iterative divide-and-merge pipeline. Starting from an initial partition, the algorithm repeatedly groups currently active supernodes,

searches for promising pairs inside each group, evaluates whether merging a pair reduces the encoding cost, and applies accepted merges to update the partition. SWeG follows this general strategy and explicitly separates dividing, merging, and encoding: shingle-based grouping restricts the candidate search space, the merging step greedily considers only within-group pairs, and the final encoding step materializes superedges and corrections [21]. In that setting, a cheap structural proxy is useful because evaluating the true correction-set saving for every possible pair is too expensive. SWeG therefore uses SuperJaccard to choose a candidate partner inside each group and then applies a saving threshold to decide whether to accept that merge [21].

Later scalable summarizers follow the same broad divide-and-merge idea but change the proxy or the representation. LDME-style methods and related scalable summarizers use locality-sensitive hashing to find promising merges efficiently [20, 27]. SLUGGER also reduces candidate search, but changes the underlying representation to a hierarchical model and performs greedy merges under that model [14]. BEAM keeps the flat correction-set representation fixed and asks a narrower question: how should the repeated merge stage be reorganized inside this flat model when the final decision is still tied to the flat correction-set objective?

A key limitation of similarity-based or proxy-based search is that the proxy is not the correction-set objective. A profile score measures whether two supernodes have related neighborhood patterns, but the actual compression effect depends on local pair capacities, supernode sizes, affected neighboring supernodes, edge counts to those neighbors, and whether sparse/dense encoding choices change after the merge. A highly ranked pair may therefore yield no encoding reduction, while a moderately ranked pair may produce positive gain if several affected pairs become cheaper after merging.

As a simple running example, first consider two singleton supernodes A and B. Suppose each has one edge to Z_1 and one edge to Z_2, where Z_1 and Z_2 each contain at least two vertices, and assume there are no internal edges and no edge between A and B. A Jaccard-style profile similarity would rank A and B very highly because their external profiles are identical. However, the external cost before merging is

$$L(A, Z_1) + L(B, Z_1) + L(A, Z_2) + L(B, Z_2) = 1 + 1 + 1 + 1 = 4,$$

After merging A and B, each merged pair is still encoded sparsely:

$$L(A \cup B, Z_1) = 2, \qquad L(A \cup B, Z_2) = 2.$$

The external cost remains 4, and the endpoint/self terms do not change under the assumptions above. Thus this highly ranked pair has zero local encoding gain. Conversely, consider two supernodes C and D, each of size 3, and four neighboring supernodes $Z_1, \ldots, Z_4$, each also of size 3. Suppose C has five edges to each of Z_1, Z_2, Z_3, while D has five edges to each of Z_1, Z_2, Z_4, again with no internal edges and no edge between C and D. Their profiles overlap only on Z_1 and Z_2, so their profile similarity is only moderate. Yet for each shared neighbor Z_i, the pre-merge cost is

$$L(C, Z_i) + L(D, Z_i) = 5 + 5 = 10,$$

whereas after merging C and D, the dense encoding cost becomes

$$L(C \cup D, Z_i) = 1 + 18 - 10 = 9.$$

The merge therefore saves one unit on each shared neighbor, and these savings accumulate over the affected neighborhood.

This example illustrates why BEAM uses lightweight profile-based proxies only to restrict the search space. The final decision for each retained candidate is based on exact local correction-set gain, which we formalize next.

2.3 Local Encoding Gain

Consider a candidate merge (X, Y) under the current partition $\mathcal{S}$. Let

$$M_{XY} = X \cup Y$$

be the merged supernode. Only encoding terms incident to X or Y can change after this merge. All other supernode-pair costs remain unchanged and cancel out when computing the gain. We define the affected scope as

$$\Omega(X, Y) = \{X, Y\} \cup \{Z \in \mathcal{S} : e(X, Z) > 0 \text{ or } e(Y, Z) > 0\}. \quad (5)$$

The external affected neighborhood excludes the two merge endpoints:

$$N_{XY} = \Omega(X, Y) \setminus \{X, Y\}. \quad (6)$$

Before merging X and Y, the local cost contributed by this scope is

$$L_{\text{before}}(X, Y) = L(X, X) + L(Y, Y) + L(X, Y) + \sum_{Z \in N_{XY}} \big(L(X, Z) + L(Y, Z)\big). \quad (7)$$

After replacing X and Y by M_{XY}, the corresponding local cost becomes

$$L_{\text{after}}(X, Y) = L(M_{XY}, M_{XY}) + \sum_{Z \in N_{XY}} L(M_{XY}, Z). \quad (8)$$

The exact local encoding gain is

$$\text{gain}(X, Y) = L_{\text{before}}(X, Y) - L_{\text{after}}(X, Y). \quad (9)$$

A positive gain means that applying this merge alone would reduce the current flat correction-set encoding cost. BEAM uses this gain as the weight of a candidate merge edge.

2.4 Merge Selection Problem

Consider one divide group Q and a candidate pair set $C_Q \subseteq Q \times Q$. Each candidate $(X, Y) \in C_Q$ has a local encoding gain $\text{gain}(X, Y)$ under the current partition. The merge-selection problem is to choose a subset $M \subseteq C_Q$ that maximizes total gain subject to a conflict constraint:

$$\max_{M \subseteq C_Q} \sum_{(X,Y) \in M} \text{gain}(X, Y)$$
$$\text{s.t.} \quad \forall X \in Q,\ X \text{ appears in at most one pair in } M.$$

This is a maximum-weight matching problem on the candidate graph induced by the current divide group. For a fixed candidate graph with fixed non-negative weights, maximum-weight matching is a polynomial-time problem [10]. In BEAM, however, this static view is only a local subproblem: both candidate edges and their weights depend on the evolving summary state. After merges are applied, supernode memberships, neighborhoods, and local encoding costs may all change, so candidate generation and gain evaluation must be repeated.

BEAM therefore uses a greedy descending-gain matching rule in each group after evaluating exact local gains for the retained candidate pairs and filtering out non-positive candidates. This choice keeps merge selection tied to the correction-set objective while avoiding the overhead of repeatedly solving exact weighted matching on many small, dynamically regenerated candidate graphs.

2.5 Parallel Gain Evaluation Challenge

The full summarization pipeline is difficult to execute directly on GPUs because the objects being processed are dynamic. Supernodes are not fixed graph vertices: their memberships, neighborhoods, candidate partners, and local encoding costs all evolve as merges are applied. Thus, a straightforward migration of the full greedy merge procedure to CUDA would still face dependencies between candidate generation, gain evaluation, conflict resolution, and partition updates.

The main computational separation in BEAM is between combinatorial merge control and repeated gain scoring. Candidate sparsification and conflict-free merge selection operate over dynamic supernodes and irregular candidate sets, while gain evaluation assigns weights to already retained candidate pairs. After candidates are scored, accepted merges must satisfy the one-merge-per-supernode constraint; applying those merges then mutates the partition and changes the inputs to later rounds. These dependencies make full-pipeline GPU execution difficult, consistent with the irregularity observed in GPU graph workloads [24].

BEAM therefore isolates local gain evaluation as the GPU-amenable scoring component inside the higher-level merge-selection algorithm. Candidate pairs and their local profile data are prepared on the host and packed into batched evaluation units. The CUDA backend evaluates these units and returns gain values, while grouping, positive-gain matching, summary update, and final correction-set encoding remain in the standard flat pipeline.

Batching changes only how retained candidate gains are evaluated, not the flat correction-set objective.

3 BEAM ALGORITHM

3.1 Overview

BEAM takes as input a graph $G = (V, E)$ and produces a flat correction-set summary under the standard correction-set semantics.

At a high level, BEAM follows the same broad divide-and-merge pattern as prior flat summarization pipelines. Active supernodes are grouped so that candidate search remains local. Within each group, BEAM constructs a sparse candidate graph, assigns each retained candidate edge an exact local correction-set gain, and selects a positive-gain conflict-free matching. Merge application is then deferred to the end of the current group block, after which the flat partition is updated. After the iterative merge process terminates, the final partition is encoded into the usual correction-set representation.

The main difference from a fine-grained merge loop is that BEAM approximates the merge search space through candidate sparsification while keeping the retained local gain objective exact. Candidate preparation and summary updates remain tied to the evolving partition, but the expensive gain evaluations are reorganized into batched units that can be executed efficiently by the current CUDA implementation without changing the merge rule.

Algorithm 1: BEAM Algorithm

Input: graph $G = (V, E)$; iteration budget T; candidate budget k.
Output: flat correction-set summary $S = (\mathcal{S}, P, C^+, C^-)$.

```
Initialize each vertex as a singleton supernode.
Initialize the active supernode set A.
for t = 1 to T do
    Q_t ← AdaptiveMultiHashDivide(A).
    Partition Q_t into group blocks B_t.
    for each group block B ∈ B_t do
        M_B ← ∅.
        for each group Q ∈ B do
            M_Q ← GreedyGainMatching(Q, k).
            M_B ← M_B ∪ M_Q.
        end for
        Apply all merges in M_B to update the flat partition.
    end for
end for
Encode the final partition into superedges P, positive corrections C+,
and negative corrections C−.
return S = (S, P, C+, C−).
```

More concretely, in iteration t, the grouping stage partitions the current active supernodes into disjoint groups $\mathcal{Q}_t = \{Q_1, \ldots, Q_r\}$. These groups are then partitioned into group blocks $\mathcal{B}_t = \{B_1, \ldots, B_s\}$. All groups inside one block are prepared, scored, and matched under the same frozen partition state; the accepted merges of that block are applied only after the block-local selection phase is complete. This separation is important for the notation below: the algorithm operates on four nested objects, namely current supernodes, a divide group Q, a group block B, and a batched gain-evaluation unit built from the retained candidate pairs of one or more groups.

BEAM differs from earlier proxy-driven flat summarization pipelines in three ways. First, a lightweight candidate proxy is used only to construct a sparse candidate graph, not to decide merges. Second, each retained candidate is scored by the exact local correction-set gain under the current partition. Third, positive-gain candidates are selected through conflict-free greedy matching, which separates gain evaluation from merge application.

3.2 Profile-Based Candidate Sparsification

Grouping is inherited from flat divide-and-merge pipelines and is used to localize candidate generation. Each iteration first partitions the active supernodes into groups that are likely to contain structurally related candidates, rather than performing a global all-pairs search. This reduces search cost and keeps merge decisions local to the current state of the flat summary.

Within each group, BEAM constructs a sparse neighborhood profile for every active supernode. The profile summarizes the current local adjacency pattern with respect to the evolving partition. Candidate generation then uses a lightweight encoding-aware proxy over these profiles to construct a bounded candidate graph by retaining up to top-k partners within the group. Section 3.3.1 defines the profile representation and this proxy-based screening step in more detail. The proxy is used only to prune the search space; exact local gain remains the final merge objective.

3.2.1 Adaptive Multi-Hash Divide. BEAM rebuilds divide groups at the beginning of each iteration from the current active partition rather than maintaining them incrementally across iterations. Each active supernode belongs to exactly one divide group in that iteration. This matters because the summary state changes after merges, so both the supernode membership and the group-local search space are iteration dependent.

The current implementation uses an adaptive multi-hash divide procedure. For iteration t, BEAM computes locality-sensitive min-hash ranks for the active supernodes over multiple hash dimensions [6, 7]. It then orders the active supernodes by one dimension at a time and recursively splits oversized buckets by additional hash dimensions until the resulting buckets satisfy a target maximum group size. If all configured hash dimensions are exhausted and a bucket is still too large, BEAM applies a deterministic fallback split. This keeps the final groups reproducible while preventing a small number of very large groups from dominating candidate generation.

This grouping stage should be distinguished from later block formation. Divide groups determine the candidate search space and the scope of within-group conflict resolution. Group blocks are a higher-level execution unit used only to batch gain evaluation and to defer merge application until the end of the current block.

3.3 Exact Local Gain Evaluation

After candidate sparsification, BEAM evaluates each retained candidate under the current summary state. Rather than re-encode the full summary for every pair, it materializes only the local information needed for the correction-set objective defined in Section 2. For implementation efficiency, these retained candidates can be packed into batched evaluation units. The discussion below focuses on how BEAM instantiates the earlier objective with sparse profiles and block-local statistics.

3.3.1 Sparse Supernode Profiles and EA-Proxy Screening. For each active supernode X, BEAM constructs a sparse neighborhood profile W_X over the current supernode partition. Each coordinate of W_X corresponds to a current supernode Z, and the value $W_X[Z]$ records the number of input edges between vertices in X and vertices currently assigned to Z. The profile is first accumulated from original graph adjacency and then aggregated according to the current supernode identifiers.

BEAM then uses these profiles to construct an encoding-aware proxy for candidate retention. The current implementation builds target-supernode buckets from the sparse profiles and generates candidate pairs only when two source supernodes co-occur in at least one target bucket. This proxy is defined according to the implemented shared-target screening rule: candidate screening follows the same pair-cost model as exact gain evaluation, but only over a cheap shared-target scope and with a lightweight direct-edge heuristic signal.

For a pair of active supernodes X and Y, let

$$B(X, Y) = \{Z \mid X \text{ and } Y \text{ co-occur in the target bucket of } Z\}.$$

For each shared target $Z \in B(X, Y)$, BEAM computes the local proxy contribution

$$\Delta_Z(X, Y) = L(X, Z) + L(Y, Z) - L(X \cup Y, Z),$$

where $L(\cdot, \cdot)$ is the same local pair-cost function used by the final correction-set encoder. The implementation accumulates only positive shared-target improvements and then adds a direct-edge heuristic signal

$$h_{\text{dir}}(X, Y) = e(X, Y).$$

The resulting proxy gain is

$$\text{proxy\_gain}(X, Y) = \sum_{Z \in B(X,Y)} \max\{0, \Delta_Z(X, Y)\} + h_{\text{dir}}(X, Y).$$

To avoid favoring large supernodes only because they participate in larger local costs, BEAM ranks candidates by a normalized proxy ratio

$$\text{proxy\_ratio}(X, Y) = \frac{\text{proxy\_gain}(X, Y)}{C^{\text{old}}(X) + C^{\text{old}}(Y) + \varepsilon},$$

where $C^{\text{old}}(X)$ and $C^{\text{old}}(Y)$ are approximate old-cost aggregates derived from the current sparse profiles, and ε is a small constant for numerical safety.

This shared-target scope is only a cheap retention scope induced by the target buckets; it is not the exact affected neighborhood used later for exact gain evaluation.

For a group Q, let $\text{top}_k(X, Q)$ denote the at most k supernodes in $Q \setminus \{X\}$ with the largest proxy ratios against X. The deduplicated group-local candidate set is

$$C_Q = \big\{\{X, Y\} \mid X, Y \in Q,\ Y \in \text{top}_k(X, Q) \text{ or } X \in \text{top}_k(Y, Q)\big\}.$$

This proxy is used only for candidate ranking and top-k retention. It is not a correctness certificate. Every retained candidate is rescored by exact local correction-set gain before greedy matching.

3.3.2 Local Statistics for Exact Gain. BEAM scores a retained candidate pair by instantiating the local pair cost from Eq. (4), with pair capacity from Eq. (1) and sparse/dense alternatives from Eqs. (2)–(3). In the implementation, the repeated work is not the definition of these quantities but the collection of the local statistics needed to evaluate them under the current partition.

For a candidate merge (X, Y), the affected external neighborhood from Eq. (6) is recovered directly from the sparse profiles as

$$N_{XY} = \big(\text{supp}(W_X) \cup \text{supp}(W_Y)\big) \setminus \{X, Y\},$$

where $\text{supp}(W_X) = \{Z \mid W_X[Z] > 0\}$. For every $Z \in N_{XY}$, the merged pair count is

$$e(M, Z) = e(X, Z) + e(Y, Z).$$

The merged self-pair uses

$$|M| = |X| + |Y|, \qquad e(M, M) = e(X, X) + e(Y, Y) + e(X, Y).$$

Using these local statistics, BEAM computes the exact local gain in Eq. (9) from the corresponding before/after costs in Eqs. (7)–(8). The unit-cost superedge term in Eq. (3) matches the implementation's encoding-cost accounting, so the resulting local decision remains the same flat correction-set objective defined in Section 2. BEAM accepts only candidates with positive local gain. The final merge decision therefore depends on the correction-set objective rather than on neighborhood similarity alone. Exact evaluation only needs the sizes and local edge counts of X, Y, and the supernodes in $\text{supp}(W_X) \cup \text{supp}(W_Y)$; unrelated supernode pairs are never rescored.

Algorithm 2: Greedy Gain Matching within One Group

Input: group Q of active supernodes; candidate budget k.
Output: conflict-free merge set $\mathcal{M}_Q$ for group Q.

for each supernode $X \in Q$ **do**
    Build sparse neighborhood profile W_X.
end for
$C \leftarrow \emptyset$.
Build target buckets from the sparse profiles in Q.
Accumulate positive EA-proxy terms and direct-edge heuristic signals for bucket-induced pairs.
for each supernode $X \in Q$ **do**
    Retain up to k partners for X by proxy ratio.
    Add candidate pairs (X, Y) to C.
end for
Deduplicate unordered candidate pairs in C.
$U \leftarrow \text{PackGainUnits}(C, \{W_X : X \in Q\})$.
$G \leftarrow \text{BatchedGainEvaluation}(U)$.
$C_{\text{pos}} \leftarrow \{(X, Y, g) \in C \mid g = G(X, Y), g > 0\}$.
Sort C_{pos} by descending gain.
$\mathcal{M}_Q \leftarrow \emptyset$; used $\leftarrow \emptyset$.
for each $(X, Y, g) \in C_{\text{pos}}$ **do**
    if $X \notin$ used **and** $Y \notin$ used **then**
        Add (X, Y) to $\mathcal{M}_Q$; add X and Y to used.
    end if
end for
return $\mathcal{M}_Q$.

Proposition 1 (Locality of gain). For a candidate merge (X, Y) under the current partition, all correction-set cost terms not incident to X or Y remain unchanged. Therefore $\text{gain}(X, Y)$ can be computed exactly from X, Y, and the affected external neighborhood N_{XY} without re-encoding the full summary. This follows because replacing X and Y by $X \cup Y$ changes only pair costs that involve one of these supernodes; all other terms cancel.

A positive value of $\text{gain}(X, Y)$ means that applying merge (X, Y) in isolation strictly decreases the current correction-set encoding cost. BEAM uses this fact as a local greedy criterion. For a conflict-free set of simultaneous merges, second-order interactions may still appear through later partition updates, so positive local gain should not be read as a global optimality certificate.

3.4 Greedy Positive-Gain Matching

After local gain evaluation, each retained candidate edge has a weight given by $\text{gain}(X, Y)$. Since each supernode can participate in at most one accepted merge in a round, merge selection can be viewed as a weighted matching problem on the candidate graph [10]. BEAM then solves this problem approximately by greedy descending-gain order: it filters out non-positive edges and constructs a conflict-free matching on the sparsified candidate graph. Conflicts are resolved within each group. Because divide groups are disjoint within an iteration, a supernode cannot appear in two different groups of the same block; endpoint conflicts therefore arise only within groups, not across groups in the same block. Across different groups in the same block, merge application is deferred until all group-local selections have been completed. Algorithm 2 summarizes this control flow, and Algorithm 3 expands the batched scoring step that converts retained candidate pairs into exact gains.

Algorithm 3: Batched Exact Gain Evaluation

Input: packed gain-evaluation unit U.
Output: gain array G.

for each candidate index i in parallel **do**
    $X \leftarrow$ `cand_src`$[i]$; $Y \leftarrow$ `cand_dst`$[i]$.
    Fetch the sparse profile slices of W_X and W_Y from the offsets stored in U.
    Form the affected neighborhood N_{XY} by Eq. (6).
    Compute merged statistics $|M|$, $e(M, M)$, and $e(M, Z)$ for all $Z \in N_{XY}$.
    Evaluate L_{before}, L_{after}, and $G[i]$ by Eqs. (7)–(9).
end for
return G.

Proposition 2 (Retained-graph matching guarantee). Fix one divide group Q and the retained positive-gain candidate graph $H_Q = (Q, C_Q^+)$ after profile-based candidate sparsification, exact local gain evaluation, and any optional threshold filtering. Let $w(X, Y) = \text{gain}(X, Y) > 0$ be the edge weight of each retained candidate. Let $\mathcal{M}_Q^g$ be the matching returned by BEAM's descending-gain greedy rule, and let $\mathcal{M}_Q^\star$ be a maximum-weight matching on the same retained graph H_Q. Then

$$\sum_{e \in \mathcal{M}_Q^g} w(e) \geq \frac{1}{2} \sum_{e \in \mathcal{M}_Q^\star} w(e).$$

Proof sketch. Consider the greedy edges in the order selected by BEAM. When a greedy edge $e = (X, Y)$ is selected, it removes from further consideration all remaining edges incident to X or Y. In the optimal matching $\mathcal{M}_Q^\star$, at most two edges can be incident to the endpoints of e: one through X and one through Y. Since BEAM processes edges in non-increasing weight order, each such blocked optimal edge has weight at most $w(e)$ at the time it is charged to e. Charging every optimal edge either to itself if it is selected by greedy or to the greedy edge that first blocks it gives a total charge of at most $2w(e)$ to each greedy edge. Therefore $w(\mathcal{M}_Q^\star) \leq 2w(\mathcal{M}_Q^g)$.

The guarantee is local to the retained candidate graph. Its scope is the conflict-resolution step after top-k sparsification, optional threshold filtering, and exact local gain evaluation have already fixed the positive-gain candidate graph. Thus, the result should be read as a bound on the loss from greedy matching relative to exact maximum-weight matching on that retained graph, rather than as a guarantee for the globally optimal flat summary.

Complexity and exactness. For a candidate pair (X, Y), BEAM computes the exact local gain in Eq. (9) over the affected neighborhood in Eq. (6), using the local pair cost from Eq. (4).

For a divide group Q, let $B_Z = \{X \in Q \mid W_X[Z] > 0\}$ be the target bucket induced by coordinate Z. The bucket-induced proxy construction enumerates co-occurring source pairs inside target buckets, with cost

$$O\left(\sum_Z |B_Z|^2\right)$$

before top-k retention and deduplication, with $O(|Q|^2)$ as the worst case when one bucket contains the whole group. This form reflects

the implementation more directly than a dense all-pairs scan: BEAM generates proxy candidates from shared target buckets rather than scoring every within-group pair by exact gain.

After top-k retention, the number of retained pairs before deduplication is at most $|Q|k$; after deduplication, let this number be c. Exact local gain evaluation then costs

$$O\left(\sum_{(X,Y)\in C} |\mathrm{supp}(W_X) \cup \mathrm{supp}(W_Y)|\right),$$

sorting positive-gain candidates costs $O(c \log c)$, and greedy conflict-free selection costs $O(c)$ using a used-supernode marker array. Thus, BEAM approximates the merge-search space through bucket-induced candidate sparsification, but not the correction-set gain of retained candidates. The greedy matching step is also approximate, but its approximation is limited and well-scoped: on a fixed retained positive-gain candidate graph, descending-gain greedy matching gives a 1/2-approximation to maximum-weight matching on that retained graph. The overall summarization algorithm remains heuristic with respect to the global flat-summary optimum because the retained candidate graph itself is sparse, local gains are recomputed after state updates, and simultaneous merges can interact through later partition changes.

3.5 State Update after Matching

Once a positive-gain matching has been selected, BEAM applies its merges to the current partition. A supernode cannot appear in more than one accepted merge within the current selection context, so the matching itself is conflict-free by construction. In the blocked schedule used by BEAM, merge application is deferred until all groups in the current group block have finished candidate scoring and group-local selection.

When a merge (X, Y) is applied, BEAM creates a new supernode $M_{XY} = X \cup Y$, removes X and Y from the active set, and inserts M_{XY}. For every neighboring supernode Z, the new incident count is

$$e(M_{XY}, Z) = e(X, Z) + e(Y, Z),$$

and the merged internal count is

$$e(M_{XY}, M_{XY}) = e(X, X) + e(Y, Y) + e(X, Y).$$

The same identities are used during gain evaluation, but here they serve a different purpose: they define the actual partition mutation and the summary state seen by the next block or iteration.

Merge application is deferred because immediate updates would entangle candidate evaluation with summary-state mutation and would eliminate the clean separation between search-space construction, edge-weight evaluation, and matching-based selection. Deferred application keeps gain evaluation separate from conflict handling. Partition updates and summary maintenance remain host-side responsibilities.

In the current implementation, sparse profiles and divide groups are rebuilt or refreshed from the updated supernode identifiers at the beginning of the next iteration rather than maintained as long-lived candidate objects across iterations. This keeps the scoring inputs consistent with the current partition and avoids carrying stale candidate pairs after merges have changed local neighborhoods.

After the iterative merge process finishes, the final flat partition is encoded using the standard correction-set representation.

4 BATCHED GAIN EVALUATION

Section 3 defined the exact local objective. This section makes explicit how BEAM turns that objective into a batched scoring workload. After deduplication, the retained candidates from one or more prepared groups are packed into a gain-evaluation unit

$$U = (\texttt{cand\_src}, \texttt{cand\_dst}, \texttt{prof\_ptr}, \texttt{prof\_nbr}, \texttt{prof\_val}, \texttt{size}, \texttt{self}),$$

where $\texttt{cand\_src}[i]$ and $\texttt{cand\_dst}[i]$ identify candidate i. The array `prof_ptr` gives the slice boundaries of each sparse profile, `prof_nbr` stores the supernode identifiers of nonzero profile entries, `prof_val` stores the corresponding edge counts, $\texttt{size}[X] = |X|$, and $\texttt{self}[X] = e(X, X)$. If several groups are packed into one batch, the host additionally stores group or batch offsets so that the returned gains can be mapped back to the corresponding group-local candidate graphs for matching.

The key point is that grouping and scoring play different roles. Grouping is a host-side search-space restriction: it determines which supernodes may become candidate partners and which candidate scores will later compete in the same greedy matching step. The batched evaluator only scores retained candidate pairs under the current partition; merge application and conflict resolution remain part of the higher-level host-side algorithm.

Algorithm 3 describes the per-candidate computation. For candidate $i = (X, Y)$, one worker reads the sparse profile slices of W_X and W_Y, traverses $\mathrm{supp}(W_X) \cup \mathrm{supp}(W_Y)$, and treats missing entries as zero. This directly yields the external counts $e(X, Z)$, $e(Y, Z)$, and $e(M, Z) = e(X, Z) + e(Y, Z)$ needed for every affected supernode Z. The same worker also computes the merged self statistics $|M| = |X| + |Y|$ and $e(M, M) = e(X, X) + e(Y, Y) + e(X, Y)$, evaluates L_{before} and L_{after}, and writes one exact gain value to $G[i]$. The work is therefore parallel across candidates but still exact with respect to the local correction-set objective.

Among the stages in the merge loop, gain evaluation is the one that admits the clearest batch structure, while candidate sparsification and summary updates remain tied to irregular control flow and state mutation. Batched evaluation therefore isolates the part of the algorithm most amenable to regular parallel execution. In the current implementation, the same packed representation can be evaluated by CPU threads or transferred to CUDA for device-side parallel scoring, whereas candidate generation, greedy matching, merge application, and final correction-set encoding remain in the host-side flat pipeline.

The benefit of the current CUDA instantiation is workload dependent. If the batched candidate set is large enough and per-candidate gain evaluation is sufficiently expensive, device execution may amortize transfer and launch overhead. If candidate sets are small or the local objective is cheap, the same overheads may limit or eliminate the benefit. The paper therefore characterizes the CUDA implementation empirically across workloads.

4.1 CUDA Mapping

The current CUDA backend treats retained candidates as independent scoring tasks rather than attempting to keep the full summary state on the device. The key mapping is that candidate scoring is parallelized across candidate pairs: each device worker receives one retained candidate together with offsets into the packed sparse-profile arrays, computes the corresponding local gain, and writes one output value. Because candidate profile lengths vary, different workers may traverse different numbers of sparse entries, so the CUDA workload is irregular even though the interface is batched.

To make this workload executable on the GPU, the host first prepares one or more groups, aggregates their sparse profiles at the current supernode level, and flattens the resulting data into contiguous arrays. Host-to-device transfer therefore moves candidate endpoints, profile offsets, profile neighbor identifiers, profile edge-count values, supernode sizes, and self-loop counts. Device-to-host transfer returns only the scored gain array. Figure 4 and Table 6 report these three component counters separately as H2D, kernel, and D2H time.

The blocked schedule also explains why the number of scoring slices and kernel launches can be large. The host repeatedly forms group blocks and invokes the CUDA scorer on those blocked workloads. If a blocked workload is still too large, the implementation may further split it into candidate-budgeted kernel chunks. The reported `Block Size` in Table 6 refers to the group-block size used by the blocked schedule, while the large `Scoring Slices` and `Kernel Launches` counts reflect repeated blocked or sub-batched scoring invocations across all iterations.

5 EXPERIMENTAL EVALUATION

5.1 Experimental Questions

Our evaluation focuses on BEAM's merge-stage organization within the flat correction-set model and separately characterizes the CUDA backend used for retained gain evaluation. We therefore organize the evaluation around three core questions and one implementation analysis question.

- **RQ1.** Does BEAM improve the speed–quality tradeoff over SWeG, the same-model flat correction-set baseline?
- **RQ2.** What tradeoff does BEAM offer relative to SLUGGER, a more expressive hierarchical summarization baseline?
- **RQ3.** Which stages dominate BEAM's runtime, and how much of the execution is concentrated in candidate sparsification and gain evaluation?
- **RQ4.** As an implementation analysis, what role does the CUDA backend play in the end-to-end BEAM implementation, and what costs arise from batched device-side gain evaluation?

These questions separate same-model comparison, cross-model context, and implementation-level analysis. This distinction matters because BEAM and SWeG produce flat correction-set summaries, whereas SLUGGER uses a hierarchical representation, while RQ4 focuses only on one scoring backend inside BEAM.

Table 2: Datasets used in the evaluation.

Dataset	Nodes	Edges	Type
Facebook(FA)	4,039	88,234	Social
Email-Enron(EM)	36,692	183,831	Email
Amazon(AM)	334,863	925,872	Network
DBLP(DB)	317,080	1,049,866	Collaboration
Youtube(YO)	1,134,890	2,987,624	Social
RoadNet(RN)	1,965,206	2,766,607	Network
Livejournal(LJ)	3,997,962	34,681,189	Social
Hollywood(HO)	1,107,243	56,375,711	Collaboration
Orkut(OR)	3,072,441	117,185,083	Network
UK-2002(U2)	18,520,343	261,787,258	Hyperlinks
UK-2005(U5)	39,459,923	783,027,125	Hyperlinks

5.2 Datasets and Baselines

We evaluate BEAM on eleven graph datasets with different sizes, sparsity levels, and graph types. Table 2 summarizes the datasets used in the evaluation. The collection includes social, email, collaboration, product, road, and hyperlink graphs. Most datasets are from SNAP [15], while the largest web graphs are from the LAW/WebGraph ecosystem [5]. The overall comparison uses the ten datasets for which all three methods have corresponding results. Orkut is excluded from the overall comparison because matching SWeG and SLUGGER results are unavailable, but it is included in BEAM's runtime-breakdown and CUDA-backend analysis.

We use the SWeG implementation released in the LDME project and the original SLUGGER implementation released by its authors. Both baselines are CPU implementations. All methods are evaluated on the same machine and with the same CPU thread setting. BEAM uses the CPU for grouping, candidate generation, greedy matching, merge application, and final encoding, and uses the NVIDIA Tesla P100 as one backend for batched gain evaluation. We report this as an end-to-end system comparison under a common hardware setting, and separately isolate the CUDA scoring cost in Section 5.7.

We compare BEAM with two baselines. **SWeG** is the primary same-model baseline because it is a scalable flat correction-set summarization method and was shown to dominate earlier lossless summarization methods in speed–quality tradeoff [2, 11, 19–21]. We start from the single-thread SWeG implementation released in the LDME project and reproduce a multithreaded version for our runs. **SLUGGER** is included as a hierarchical contrast baseline. Since SLUGGER uses a different summary representation, this comparison should be interpreted as cross-model context rather than a same-semantics comparison. For SLUGGER, we use the original code released with the paper.

All methods are evaluated under the same hardware and software environment. The experiments run on a dual-socket server with two Intel Xeon Silver 4316 CPUs at 2.30 GHz, providing 40 physical cores and 80 hardware threads in total, and an NVIDIA Tesla P100 GPU. Unless otherwise stated, all methods are run for 20 iterations. The reported BEAM-CUDA runs use top-$k = 16$, a group-block size of 64, and a candidate batch budget of 65,536. In this section, "BEAM" refers to the implementation using batched CUDA gain evaluation.

5.3 Metrics

We report two end-to-end metrics: relative output size and total runtime. Relative output size measures the size of the compressed representation normalized by the number of original graph edges:

$$\text{RelSize} = \frac{|P| + |C^+| + |C^-|}{|E|},$$

where P is the superedge set and C^+ and C^- are the positive and negative correction sets. Lower relative size indicates better compression. Runtime is the total wall-clock execution time, where lower is better.

For BEAM, we further report stage-level runtime. We separate divide time, merge time, encode time, candidate-generation time, gain-scoring time, merge share, and scoring share. Here merge share and scoring share denote the fractions of total runtime spent in merge-related work and in the combined prepare-and-exact-scoring portion of the merge stage, respectively. For the blocked implementation, **Prepare Wall** denotes the CPU-side wall-clock time spent preparing retained candidates before exact scoring, including sparse-profile refresh, target-bucket construction, EA-proxy accumulation, top-k retention, candidate deduplication, and construction of the retained candidate data consumed by the scoring backend. **Gain Scoring Wall** denotes only the wall-clock time of the exact-gain evaluator: the CPU exact-gain evaluator time for the CPU backend and the CUDA scoring wall time for the CUDA backend. For CUDA, we also report H2D, kernel, D2H, scoring slices, and kernel launches as component counters rather than as a strictly additive decomposition of CUDA wall-clock time.

5.4 RQ1: Same-Model Comparison with SWeG

Table 3, Figure 1, and Figure 2 compare BEAM with SWeG on the ten common datasets. This is the primary comparison because both methods use the flat correction-set representation.

In this end-to-end implementation comparison, BEAM is faster than SWeG on all ten common datasets. The geometric mean speedup over SWeG is 2.69×. In terms of compression, BEAM produces smaller relative output size on all ten common datasets. These results show that BEAM improves the speed–quality tradeoff within the same summary model: its candidate-sparse merge-stage organization reduces runtime while its exact local gain scoring preserves or improves compression quality across the common datasets.

BEAM relies on adaptive divide grouping and top-k profile-based candidate retention to restrict the candidate graph. This design reduces search cost while still preserving the same-model speed–quality tradeoff against SWeG under the reported settings.

5.5 RQ2: Cross-Model Comparison with SLUGGER

We also compare BEAM with SLUGGER to provide cross-model context against a more expressive hierarchical summarization method. BEAM outputs a flat correction-set summary, whereas SLUGGER uses a hierarchical representation.

Under the same experimental setting, BEAM is faster than SLUGGER on nine of the ten common datasets; the exception is RoadNet. The geometric mean speedup over SLUGGER is 2.29×. Compression quality remains mixed. BEAM obtains smaller relative size on EM, AM, DB, YO, RN, LJ, U2, and U5, while SLUGGER obtains smaller relative size on FA and HO. This result is consistent with the difference between the two representations: SLUGGER's hierarchy can express some structures more compactly, while BEAM keeps the flat representation and focuses on repeated merge-stage organization. The comparison is therefore best read as cross-model context, showing that a flat correction-set method can remain competitive in compression while offering substantially lower runtime in this setting.

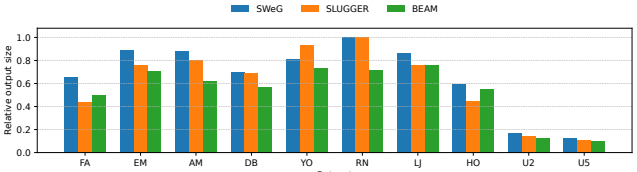

Figure 1: RQ1–RQ2: Relative output size on common datasets. Relative size is the compressed representation size divided by the number of original edges; lower is better. BEAM improves over SWeG on all ten common datasets, while SLUGGER remains a hierarchical contrast with mixed compression tradeoffs.

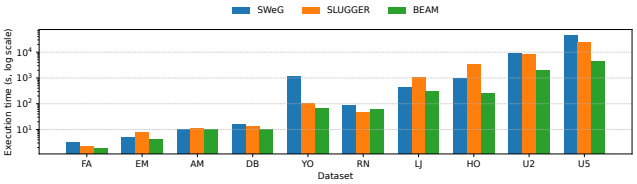

Figure 2: RQ1–RQ2: Execution time on common datasets under 20 iterations. The y-axis uses a logarithmic scale; lower is better. BEAM is faster than SWeG on all ten common datasets and faster than SLUGGER on nine of them.

5.6 RQ3: Runtime Concentration in the Merge Stage

Figure 3 and Table 4 show that merge-related work accounts for a large fraction of BEAM's runtime and dominates on most datasets in the batch-ea-blocked CUDA configuration. Merge share ranges from 50.0% to 97.9% of total runtime. The combined prepare-and-gain-scoring share ranges from 21.3% to 79.4%, while gain scoring alone ranges from 2.0% to 19.0%. Prepare work is often the largest visible component, while exact gain scoring remains a persistent but smaller part of merge cost across datasets. This pattern supports the merge-stage bottleneck claim: the main scalability challenge in BEAM is not final correction-set encoding, but the repeated construction, scoring, selection, and application of candidate merges under a changing partition.

At the same time, the breakdown shows that gain evaluation alone is not the only bottleneck. Prepare work, including sparse-profile refresh, target-bucket construction, EA-proxy accumulation, top-k retention, deduplication, and retained-candidate packing, can

Table 3: RQ1–RQ2: Overall comparison on the ten common datasets. Lower relative size and lower runtime are better.

Dataset	SWeG Rel. Size	SWeG Time (s)	SLUGGER Rel. Size	SLUGGER Time (s)	BEAM Rel. Size	BEAM Time (s)	Speedup vs SWeG	Speedup vs SLUGGER
FA	0.657	3.27	**0.442**	2.31	0.497	**1.87**	1.75×	1.24×
EM	0.891	5.03	0.763	7.63	**0.711**	**3.99**	1.26×	1.91×
AM	0.879	10.55	0.802	11.11	**0.619**	**10.35**	1.02×	1.07×
DB	0.698	15.34	0.690	13.82	**0.571**	**10.11**	1.52×	1.37×
YO	0.815	1127.86	0.935	103.32	**0.735**	**68.90**	16.37×	1.50×
RN	0.999	87.65	1.000	**46.89**	**0.716**	62.36	1.41×	0.75×
LJ	0.865	435.19	0.764	1061.45	**0.756**	**299.03**	1.46×	3.55×
HO	0.598	987.03	**0.445**	3525.73	0.549	**263.78**	3.74×	13.37×
U2	0.173	9442.87	0.144	8071.43	**0.126**	**2024.26**	4.66×	3.99×
U5	0.130	45681.93	0.110	24170.22	**0.101**	**4585.81**	9.96×	5.27×
Geo. mean speedup	–	–	–	–	–	–	**2.69**×	**2.29**×

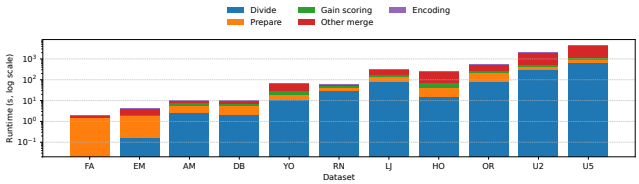

Figure 3: RQ3: Runtime breakdown of BEAM. The stacked bars separate divide, prepare, gain scoring, other merge work, and final encoding. Merge-related stages dominate total runtime on most datasets.

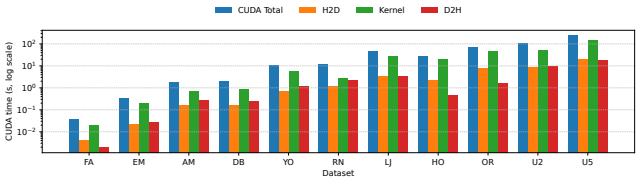

Figure 4: RQ4: CUDA scoring-path counters for BEAM's batched gain evaluation. The grouped bars report CUDA Total wall time together with H2D, kernel, and D2H component counters. The component counters are not expected to sum to CUDA Total when stream overlap is enabled.

dominate on several datasets. This observation explains why GPU acceleration of gain scoring improves only one component of the pipeline and why the surrounding blocked merge machinery remains important for end-to-end performance.

5.7 RQ4: CUDA Gain-Evaluation Cost

To separate the benefit of BEAM's merge-stage organization from the benefit of the CUDA scoring backend, we compare two implementations of the same batch-ea-blocked algorithm: one using CPU gain scoring and one using CUDA gain scoring. Both configurations use the same divide procedure, retained candidate generation, greedy positive-gain matching, deferred merge updates, and final correction-set encoding. The two backends produce identical summary outputs in our runs, including cost ratio, encoding cost, supernode count, and the final correction-set edge counts; only the execution backend for retained gain evaluation changes.

Table 5 shows that CUDA improves end-to-end runtime on 9 of the 11 datasets, with a geometric mean end-to-end speedup of 1.19×. The benefit is workload dependent. On small datasets such as FA and EM, CUDA overhead outweighs device-side scoring gains, leading to slowdowns. On larger or more scoring-intensive datasets such as AM, DB, RN, and LJ, CUDA provides clearer end-to-end acceleration. CUDA therefore acts as a backend for retained-candidate scoring when that workload is large enough, while end-to-end gains still depend mainly on BEAM's candidate-sparse merge organization.

Figure 4 and Table 6 further characterize where time is spent inside the CUDA scoring path. We report CUDA Total as the wall-clock time of the high-level CUDA scoring call, and we separately report H2D, kernel, D2H, scoring slices, and kernel launches as component counters. Because CUDA streams may overlap transfer and kernel execution, these component counters are diagnostic measurements and may not sum to CUDA Total.

The cost balance varies across datasets. On the largest common workload, U5, kernel execution takes 136.373s and host-to-device transfer adds 19.298s. U5 also issues 1,618,975 scoring slices and 1,607,709 kernel launches, reflecting the blocked execution schedule rather than a single monolithic GPU run. BEAM repeatedly forms group blocks and invokes the CUDA backend on these blocked workloads; large blocked workloads may be further split by the candidate batch budget. Thus, the reported slice and launch counts should be interpreted as repeated blocked or sub-batched gain-scoring invocations.

Across datasets, CUDA gain-evaluation time accounts for 2.0%–19.0% of end-to-end BEAM runtime. Together with Table 5, this shows that CUDA accelerates retained gain scoring when the workload is large enough, while the end-to-end performance also depends on CPU-side grouping, candidate generation, prepare work, greedy matching, merge application, and final encoding. These results position CUDA as a scoring backend within BEAM's candidate-sparse, encoding-aware merge-stage organization.

5.8 Parameter Sensitivity

We evaluate BEAM's sensitivity to its main pruning parameter, top-k, on FA, EM, AM, DB, and YO using the same batch-ea-blocked CUDA configuration as in the main experiments. Figure 5 reports geometric-mean runtime, relative output size, and retained candidate count normalized to the default $k = 16$. Smaller k reduces retained candidates but degrades compression, while increasing k from 16 to 32 gives only a small relative-size improvement at the

Table 4: RQ3: BEAM runtime breakdown across datasets.

Dataset	Total (s)	Divide (s)	Merge (s)	Encode (s)	Prepare (s)	Gain Scoring (s)	Merge Share	Prep.+Gain Share	Gain Share	Candidates
FA	1.87	0.02	1.83	0.02	0.36	0.04	97.9%	21.3%	2.0%	11,713
EM	3.99	0.17	3.79	0.04	1.82	0.32	94.8%	53.7%	8.0%	359,890
AM	10.35	2.65	7.48	0.22	2.32	1.82	72.3%	40.1%	17.6%	299,875
DB	10.11	2.18	7.68	0.24	2.47	1.92	76.0%	43.5%	19.0%	380,762
YO	68.90	11.33	56.84	0.71	38.33	10.56	82.5%	70.9%	15.3%	13,548,785
RN	62.36	30.24	31.20	0.88	8.55	11.53	50.0%	32.2%	18.5%	862,286
LJ	299.03	81.19	208.04	9.73	112.69	44.28	69.6%	52.5%	14.8%	4,175,972
HO	263.78	15.69	236.21	11.87	183.70	25.65	89.5%	79.4%	9.7%	7,659,141
OR	581.63	82.22	467.60	31.77	273.60	69.38	80.4%	59.0%	11.9%	1,801,651
U2	2024.26	303.31	1679.29	41.29	1465.70	99.32	83.0%	77.3%	4.9%	310,296,688
U5	4585.81	651.09	3789.20	144.87	3280.82	231.95	82.6%	76.6%	5.1%	823,853,836

Table 5: End-to-end comparison between CPU and CUDA exact-gain scoring backends under the same BEAM batch-ea-blocked pipeline. Both configurations use the same adaptive divide procedure, EA-proxy candidate retention, greedy matching, merge update, and final encoding; only the retained exact-gain scoring backend changes. CUDA scoring wall share is computed as `cuda_total_ms / runtime_end_to_end_ms`.

Dataset	CPU-backend E2E Time (s)	CUDA-backend E2E Time (s)	E2E Speedup	CUDA Scoring Wall Share
FA	0.58	1.87	0.31×	2.0%
EM	3.42	3.99	0.86×	8.0%
AM	20.38	10.35	1.97×	17.6%
DB	19.09	10.11	1.89×	19.0%
YO	90.44	68.90	1.31×	15.3%
RN	163.27	62.36	2.62×	18.5%
LJ	397.39	299.03	1.33×	14.8%
HO	276.52	263.78	1.05×	9.7%
OR	663.50	581.63	1.14×	11.9%
U2	2254.13	2024.26	1.11×	4.9%
U5	4993.09	4585.81	1.09×	5.1%

Table 6: RQ4: CUDA scoring-path measurements for BEAM. CUDA Total is the wall-clock time of the high-level CUDA scoring path. H2D, kernel, and D2H are component counters and may not sum to CUDA Total because CUDA streams can overlap.

Dataset	Candidates	Block Size	CUDA Total (s)	H2D (s)	Kernel (s)	D2H (s)	Scoring Slices	Kernel Launches	Total Time (s)	CUDA Share
FA	11,713	64	0.037	0.004	0.020	0.002	168	168	1.87	2.0%
EM	359,890	64	0.319	0.022	0.188	0.027	2,774	2,774	3.99	8.0%
AM	299,875	64	1.822	0.162	0.670	0.267	28,160	28,148	10.35	17.6%
DB	380,762	64	1.922	0.157	0.842	0.248	26,031	26,025	10.11	19.0%
YO	13,548,785	64	10.550	0.652	5.814	1.106	116,126	115,821	68.90	15.3%
RN	862,286	64	11.512	1.139	2.606	2.142	229,609	229,476	62.36	18.5%
LJ	4,175,972	64	44.245	3.231	27.546	3.302	339,114	336,759	299.03	14.8%
HO	7,659,141	64	25.648	2.256	18.694	0.449	40,442	40,442	263.78	9.7%
OR	1,801,651	64	69.360	7.302	46.504	1.658	138,616	136,932	581.63	11.9%
U2	310,296,688	64	99.230	8.474	52.065	9.485	928,803	922,330	2024.26	4.9%
U5	823,853,836	64	231.791	19.298	136.373	17.071	1,618,975	1,607,709	4585.81	5.1%

cost of many more retained candidates. We therefore use $k = 16$ as a fixed default.

We also tested group-block sizes in $\{32, 64, 128, 256\}$ and CUDA candidate-batch budgets from 4,096 to 65,536. Relative output size was essentially unchanged in these sweeps, and runtime differences mainly reflected batching and launch overhead. We use block size 64 and candidate budget 65,536 as stable defaults rather than tuning them per dataset.

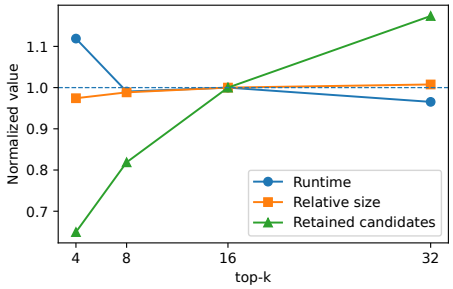

Figure 5: Sensitivity to top-k on five datasets. Values are geometric means normalized to the default $k = 16$. Increasing k from 16 to 32 slightly improves relative output size but retains more candidates, so BEAM uses $k = 16$ as a fixed stable default.

6 RELATED WORK

Graph summarization is closely related to graph compression and compact graph representation. General graph-compression methods reduce storage through vertex ordering, reference encoding, interval encoding, locality-aware layouts, recursive partitioning, and compressed graph processing [1, 3–5, 8, 9, 16, 23]. These methods usually encode a fixed graph compactly. In contrast, graph summarization constructs a smaller structural representation, such as supernodes, superedges, and correction information, that can reconstruct or approximate the original graph [17, 19].

BEAM is closest to flat correction-set graph summarization. Prior flat methods, including SWeG, use divide-and-merge pipelines to restrict candidate search and construct compact summaries under flat correction-set semantics [2, 12, 13, 21]. BEAM keeps the same representation family but changes the merge-stage organization: lightweight profiles are used only to sparsify candidate search, while retained candidates are scored by exact local correction-set gain. Hierarchical summarizers such as SLUGGER use nested supernodes and therefore represent a different model class [14, 25]; we use them as cross-model baselines rather than same-model competitors.

Scalable and parallel graph-processing systems optimize traversal, filtering, sparse linear algebra, and frontier-style computation over relatively fixed graph representations [18, 22–24, 26]. BEAM addresses a different bottleneck: in flat graph summarization, the objects being processed are evolving supernodes, candidate pairs, local neighborhoods, and merge decisions. BEAM therefore isolates exact local gain evaluation as a batched scoring step while keeping grouping, matching, and partition updates in the host-side summarization pipeline.

7 CONCLUSION

We presented BEAM, an encoding-aware greedy merging algorithm for scalable flat correction-set graph summarization. BEAM preserves the flat summary model but reorganizes the repeated merge stage by separating cheap candidate sparsification from exact local correction-set gain evaluation. This design enables batched retained-candidate scoring while keeping conflict-free matching and partition updates in the standard flat pipeline. Experiments on ten common datasets show that BEAM is faster than SWeG on all datasets and faster than SLUGGER on nine, with geometric mean speedups of 2.69× and 2.29×, respectively. BEAM also improves relative output size over SWeG on all common datasets, while the comparison with hierarchical SLUGGER remains mixed.

ACKNOWLEDGMENTS

This work was supported in part by the National Key R&D Program of China under Grant No. 2023YFB3002702 and in part by the Creative Research Groups Program of the National Natural Science Foundation of China under Grant No. 62321003. Corresponding author: Guoqing Xiao.

REFERENCES

[1] Alberto Apostolico and Guido Drovandi. 2009. Graph Compression by BFS. *Algorithms* 2, 3 (2009), 1031–1044.
[2] Maham Anwar Beg, Muhammad Ahmad, Arif Zaman, and Imdadullah Khan. 2018. Scalable Approximation Algorithm for Graph Summarization. In *Advances in Knowledge Discovery and Data Mining - 22nd Pacific-Asia Conference, PAKDD 2018, Melbourne, VIC, Australia, June 3-6, 2018, Proceedings, Part III (Lecture Notes in Computer Science)*, Dinh Q. Phung, Vincent S. Tseng, Geoffrey I. Webb, Bao Ho, Mohadeseh Ganji, and Lida Rashidi (Eds.). Springer, 502–514.
[3] Maciej Besta and Torsten Hoefler. 2018. Survey and Taxonomy of Lossless Graph Compression and Space-Efficient Graph Representations. *CoRR* abs/1806.01799 (2018).
[4] Paolo Boldi, Marco Rosa, Massimo Santini, and Sebastiano Vigna. 2011. Layered label propagation: a multiresolution coordinate-free ordering for compressing social networks. In *Proceedings of the 20th International Conference on World Wide Web* (Hyderabad, India) *(WWW '11)*. Association for Computing Machinery, New York, NY, USA, 587–596.
[5] Paolo Boldi and Sebastiano Vigna. 2004. The WebGraph Framework I: Compression Techniques. In *Proceedings of the 13th International Conference on World Wide Web, WWW 2004, New York, NY, USA, May 17-22, 2004*, Stuart I. Feldman, Mike Uretsky, Marc Najork, and Craig E. Wills (Eds.). ACM, 595–602.
[6] Andrei Z. Broder. 1997. On the resemblance and containment of documents. In *Compression and Complexity of SEQUENCES 1997, Positano, Amalfitan Coast, Salerno, Italy, June 11-13, 1997, Proceedings*, Bruno Carpentieri, Alfredo De Santis, Ugo Vaccaro, and James A. Storer (Eds.). IEEE, 21–29.
[7] Andrei Z. Broder, Moses Charikar, Alan M. Frieze, and Michael Mitzenmacher. 2000. Min-Wise Independent Permutations. *J. Comput. Syst. Sci.* 60, 3 (2000), 630–659.
[8] Flavio Chierichetti, Ravi Kumar, Silvio Lattanzi, Michael Mitzenmacher, Alessandro Panconesi, and Prabhakar Raghavan. 2009. On Compressing Social Networks. In *Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD 2009, Paris, France, June 28 - July 1, 2009*. ACM, 219–228.
[9] Laxman Dhulipala, Igor Kabiljo, Brian Karrer, Giuseppe Ottaviano, Sergey Pupyrev, and Alon Shalita. 2016. Compressing Graphs and Indexes with Recursive Graph Bisection. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD 2016, San Francisco, CA, USA, August 13-17, 2016*. ACM, 1535–1544.
[10] Harold N. Gabow. 1983. An Efficient Reduction Technique for Degree-Constrained Subgraph and Bidirected Network Flow Problems. In *Proceedings of the 15th Annual ACM Symposium on Theory of Computing, STOC 1983, Boston, MA, USA, April 25-27, 1983*. ACM, 448–456.
[11] Kifayat-Ullah Khan, Waqas Nawaz, and Young-Koo Lee. 2015. Set-based Approximate Approach for Lossless Graph Summarization. *Computing* 97, 12 (2015), 1185–1207.
[12] Jihoon Ko, Yunbum Kook, and Kijung Shin. 2020. Incremental Lossless Graph Summarization. In *KDD '20: The 26th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Virtual Event, CA, USA, August 23-27, 2020*, Rajesh Gupta, Yan Liu, Jiliang Tang, and B. Aditya Prakash (Eds.). ACM, 317–327.
[13] Kyuhan Lee, Hyeonsoo Jo, Jihoon Ko, Sungsu Lim, and Kijung Shin. 2020. SSumM: Sparse Summarization of Massive Graphs. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (Virtual Event, CA, USA) *(KDD '20)*. Association for Computing Machinery, New York, NY, USA, 144–154.
[14] Kyuhan Lee, Jihoon Ko, and Kijung Shin. 2022. SLUGGER: Lossless Hierarchical Summarization of Massive Graphs. In *38th IEEE International Conference on Data Engineering, ICDE 2022, Kuala Lumpur, Malaysia, May 9-12, 2022*. IEEE, 472–484.
[15] Jure Leskovec and Andrej Krevl. 2014. SNAP Datasets: Stanford Large Network Dataset Collection. http://snap.stanford.edu/data. Accessed for benchmark

graph datasets.

[16] Yongsub Lim, U Kang, and Christos Faloutsos. 2014. Slashburn: Graph compression and mining beyond caveman communities. *IEEE Transactions on Knowledge and Data Engineering* 26, 12 (2014), 3077–3089.

[17] Yike Liu, Tara Safavi, Abhilash Dighe, and Danai Koutra. 2018. Graph Summarization Methods and Applications: A Survey. *ACM Comput. Surv.* 51, 3 (2018), 62:1–62:34.

[18] Duane Merrill, Michael Garland, and Andrew S. Grimshaw. 2012. Scalable GPU graph traversal. In *Proceedings of the 17th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, PPOPP 2012, New Orleans, LA, USA, February 25-29, 2012*, J. Ramanujam and P. Sadayappan (Eds.). ACM, 117–128.

[19] Saket Navlakha, Rajeev Rastogi, and Nisheeth Shrivastava. 2008. Graph summarization with bounded error. In *Proceedings of the ACM SIGMOD International Conference on Management of Data, SIGMOD 2008, Vancouver, BC, Canada, June 10-12, 2008*, Jason Tsong-Li Wang (Ed.). ACM, 419–432.

[20] Hojin Seo, Kisung Park, Yongkoo Han, Hyunwook Kim, Muhammad Umair, Kifayat Ullah Khan, and Young Koo Lee. 2020. An Effective Graph Summarization and Compression Technique for a Large-scaled Graph. *J. Supercomput.* 76, 10 (2020), 7906–7920.

[21] Kijung Shin, Amol Ghoting, Myunghwan Kim, and Hema Raghavan. 2019. SWeG: Lossless and Lossy Summarization of Web-Scale Graphs. In *The World Wide Web Conference, WWW 2019, San Francisco, CA, USA, May 13-17, 2019*, Ling Liu, Ryen W. White, Amin Mantrach, Fabrizio Silvestri, Julian J. McAuley, Ricardo Baeza-Yates, and Leila Zia (Eds.). ACM, 1679–1690.

[22] Julian Shun and Guy E. Blelloch. 2013. Ligra: a lightweight graph processing framework for shared memory. In *ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, PPoPP '13, Shenzhen, China, February 23-27, 2013*, Alex Nicolau, Xiaowei Shen, Saman P. Amarasinghe, and Richard W. Vuduc (Eds.). ACM, 135–146.

[23] Julian Shun, Laxman Dhulipala, and Guy E. Blelloch. 2015. Smaller and Faster: Parallel Processing of Compressed Graphs with Ligra+. In *2015 Data Compression Conference, DCC 2015, Snowbird, UT, USA, April 7-9, 2015*. IEEE, 403–412.

[24] Yangzihao Wang, Andrew A. Davidson, Yuechao Pan, Yuduo Wu, Andy Riffel, and John D. Owens. 2016. Gunrock: a high-performance graph processing library on the GPU. In *Proceedings of the 21st ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, PPoPP 2016, Barcelona, Spain, March 12-16, 2016*, Rafael Asenjo and Tim Harris (Eds.). ACM, 11:1–11:12.

[25] Rui Yan, Zi Yuan, Xiaojun Wan, Yan Zhang, and Xiaoming Li. 2012. Hierarchical Graph Summarization: Leveraging Hybrid Information through Visible and Invisible Linkage. In *Advances in Knowledge Discovery and Data Mining*, Pang-Ning Tan, Sanjay Chawla, Chin Kuan Ho, and James Bailey (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 97–108.

[26] Carl Yang, Aydin Buluç, and John D. Owens. 2022. GraphBLAST: A High-Performance Linear Algebra-based Graph Framework on the GPU. *ACM Trans. Math. Softw.* 48, 1 (2022), 1:1–1:51.

[27] Quinton Yong, Mahdi Hajiabadi, Venkatesh Srinivasan, and Alex Thomo. 2021. Efficient Graph Summarization using Weighted LSH at Billion-Scale *(SIGMOD '21)*. Association for Computing Machinery, New York, NY, USA, 2357–2365.